

EgoLife Two-User QA Pipeline

Exact current pipeline for generate_video_qa_loop, including baseline, discovery, gates, retries, and output artifacts.

Area	Source anchor
Main loop	video_qa_loop.py:620 generate_video_qa_loop
Mode argument	video_qa_loop.py:1137 add_video_loop_args
Baseline generator	prompts.py:239 build_video_generation_prompt
Discovery planner	prompts.py:337 build_relation_discovery_prompt
Discovery MCQ writer	prompts.py:378 build_relation_mcq_prompt
Reviewer prompt	prompts.py:425 build_judger_prompt
Answerability prompt	prompts.py:473 build_answerability_prompt
Judge gate	video_qa_loop.py:367 judge_gate
Answerability gate	video_qa_loop.py:331 answerability_gate
Schema validation	schema.py:81 validate_qa_item

This document describes the current code path, not an idealized design. Baseline and discovery differ only in the generation stage; after a QA is produced, both modes use the same schema validation, reviewer, answerability evaluations, strict validation, traces, accepted output, and rejected output.

1. Pipeline Summary

Evidence packet	question_type	mode branch
baseline	direct generator	QA JSON
discovery	relation planner	MCQ generator
normalize + schema	reviewer + judge_gate	answerability gate
strict validation	accepted QA	or retry with feedback

The core loop reads one evidence packet at a time. For each packet, it selects a target question type, prepares media paths, generates a QA candidate, validates the candidate, asks a VLM reviewer to judge it, tests answerability under restricted video conditions, and accepts the row only if every gate passes.

Exact stage sequence

Step	Exact action	Key code
1	Create runner, output streams, resume state, and target type counts.	generate_video_qa_loop
2	Read each evidence packet from evidence JSONL.	iter_jsonl(evidence_path)
3	Choose question_type: commonality or difference.	choose_question_type / fixed schedule
4	Resolve media paths for all packet clips.	media_for_clips
5A	Baseline: build one direct generation prompt.	build_video_generation_prompt
5B	Discovery: build relation discovery prompt and parse selected relation.	build_relation_discovery_prompt
6B	Discovery only: build MCQ prompt from selected relation.	build_relation_mcq_prompt
7	Call runner.generate for the generator prompt.	runner.generate
8	Extract JSON and attach metadata/traces.	extract_json_object, complete_generator_metadata
9	Run basic schema validation.	validate_qa_item
10	Run VLM reviewer and deterministic judge gate.	build_judger_prompt, judge_gate
11	Run answerability evaluations on restricted media conditions.	run_answerability_eval
12	Run deterministic answerability gate.	answerability_gate
13	Build final review and run strict validation.	build_review_from_gates, validate_qa_item(strict_review=True)
14	Accept row or retry with feedback until max_attempts.	accepted / rejected streams

2. Generation Modes

The command-line mode is selected with `--generation-mode`. The valid values are `baseline`, `clip_guided`, and `discovery`. The user asked for baseline comparison, so this section focuses on baseline versus discovery; `clip_guided` is included as a side note.

Aspect	baseline	discovery
Generator calls per attempt	One VLM call: direct MCQ generation.	Two VLM calls: relation discovery, then MCQ generation.
First generation task	Find relation and write question in one pass.	List 3-5 cross-user information needs and select one relation.
Second generation task	None.	Convert selected_relation into one five-option MCQ.
Prompt style	More prescriptive. Includes example and compact design rules.	More open-ended planning, then relation-conditioned writing.
Prompt trace	generation, judge, answerability.	discovery, generation, judge, answerability.
Feedback use	Feedback goes back into the direct generator prompt.	Feedback goes into the next discovery prompt and the MCQ prompt.
Cost and latency	Lower.	Higher, because every attempt adds a planning call.
Same downstream gates?	Yes: schema, reviewer, answerability, strict validation.	Yes: same gates after generation.

Baseline mode

Baseline calls `build_video_generation_prompt` once per attempt. The model must inspect the raw videos, discover a useful cross-user dependency, write the MCQ, provide evidence, explain why single users are insufficient, and return the final QA JSON in one pass.

Discovery mode

Discovery splits the generator into two calls. First, `build_relation_discovery_prompt` asks the model not to write the MCQ yet, but to list 3-5 possible cross-user information needs and select one relation. Second, `build_relation_mcq_prompt` turns that selected_relation into the final five-option MCQ.

Clip-guided side note

`clip_guided` uses the direct baseline generator path, but injects CLIP retrieval hints as attention guidance. In the current active path, discovery mode does not inject the CLIP guidance block.

3. Generated QA Shape

Both baseline and discovery ultimately ask for the same VIDEO_GENERATION_SCHEMA. The important fields are not just the question and answer; the generator must also explain two-user necessity and provide evidence that later gates can inspect.

Field group	Fields	Purpose
Question	qa_id, question_type, question, options, correct, answer, category	The visible MCQ and its broad topic label.
Users and evidence	required_users, evidence, referred_timestamps, per_user_evidence_claims	Names the users and the visual facts each user contributes.
Answerability rationale	single_user_answerability, combined_answerability, why_two_users_needed	States why each single user is insufficient and why combined users are sufficient.
Generator self-check	generator_rationale, review.generator_self_check	Provides the generator's own explanation before real gates run.

After parsing, complete_generator_metadata fills or normalizes some fields. For example, it sets question_type and required_users from the packet, defaults category to environmental_interaction if missing, normalizes correct/answer, backfills weak single_user_answerability text, and attaches generation_trace and human_audit.

4. Validation And Gates

Gate	What runs	Pass condition	Failure feedback
Basic schema	validate_qa_item(qa)	Required fields, five options, correct answer, required_users, review object, answerability text.	Schema errors to fix: ...
Reviewer	build_judger_prompt -> runner.generate -> judge_gate	All structured blocking checks exist, all are PASS, and blocking_failures is empty.	feedback_to_generator, gate reason, or Judger rejected the question.
Answerability	run_answerability_eval -> answerability_gate	combined_all_users picks the correct answer; every single/proper subset is insufficient or wrong.	Answerability gate failed: ...
Strict validation	validate_qa_item(qa, strict_review=True)	Final review, judge gate, answerability gate, traces, human audit, and schema validation all pass.	Strict validation errors: ...

Reviewer checks

- first_person_naturalness: the question should sound natural from a first-person memory or AR-assistant perspective.
- agent_perspective: the question/options should not sound like dataset annotations and should not mention video/frame/camera/timestamp language.
- source_scope: the answer should come from provided raw videos and metadata only.
- question_type_semantics: commonality and difference questions should match their intended semantics.
- multi_video_necessity: the central check. The QA must require a situated dependency across required users' videos.
- visual_grounding: the correct option and evidence claims must be grounded in visible moments.
- mcq_option_quality: exactly five plausible options and exactly one correct answer.
- gaze_safety: no invented precise gaze-to-object claim when 2D gaze is unavailable.
- human_audibility: enough user/video/time evidence exists for later human inspection.

Answerability conditions

Condition type	When created	Purpose
single_user::	One for each required user.	Checks whether any one user's media alone leaks the correct answer.
proper_subset::	Only when more than two users are required.	Checks whether a subset smaller than all required users is enough.
combined_all_users::	Always once per QA.	Checks whether all required users together can answer correctly.

The answerability gate is intentionally behavioral. It does not trust the generator's own single_user_answerability text. It asks the model to answer using restricted media. If a single user or proper subset can choose the correct answer, the QA fails even if the reviewer passed it.

5. Retry Behavior

Each evidence packet gets up to `max_attempts` attempts. The default is 3. A failed attempt becomes feedback for the next attempt. This is why the pipeline can recover from malformed JSON, weak evidence, reviewer rejection, answer leakage, or strict validation problems.

Failure stage	Example feedback	Next attempt behavior
Discovery JSON	Discovery output was not valid JSON: ...	Discovery mode asks the planner again.
Generator JSON	Generator output was not valid JSON: ...	Generator is asked to return valid JSON.
Schema	Schema errors to fix: ...	Generator must fix structural fields.
Reviewer	<code>feedback_to_generator</code> or gate reason	Generator must fix semantic/style/reviewer failures.
Answerability	Answerability gate failed: single/subset condition answered correctly: ...	Generator should choose or phrase a QA that no single user can answer.
Strict validation	Strict validation errors: ...	Generator/review metadata must satisfy final required fields.

In baseline, feedback is injected into the direct generator prompt. In discovery, feedback is injected into the discovery planner and then into the MCQ writer, so the next attempt can choose a different relation instead of merely rewording the same flawed one.

6. Output Artifacts And Traceability

Artifact	What it stores
--output	Accepted QA rows. These are final rows that passed all gates.
--rejected-output	Per-packet rejected attempts, final rejection reason, and generation_trace.
--prompts-output	Every prompt row with stage, mode, attempt, media paths, and condition media where applicable.
--intermediate-output	Accepted attempt traces, including parsed outputs and gate details.
generation_trace	Per-QA full attempt trace, attached to the accepted or rejected QA.
human_audit	Human-readable media/evidence packet summary for later review.

The most useful debugging artifact is --prompts-output. It records the exact rendered prompts sent to the model for each stage. In baseline mode, a normal attempt produces generation, judge, and answerability rows. In discovery mode, a normal attempt adds a discovery row before generation.

Prompt rows by mode

Mode	Prompt stages written per attempt
baseline	generation -> judge -> answerability for each condition
clip_guided	generation with CLIP hints -> judge -> answerability for each condition
discovery	discovery -> generation -> judge -> answerability for each condition

Accepted row requirement

- The generated QA parses as JSON.
- Basic schema validation passes.
- The VLM reviewer output passes judge_gate.
- Combined-all-users answerability selects the correct answer.
- Every single-user or proper-subset answerability condition is insufficient or wrong.
- Strict validation passes after the final review object is attached.

7. Practical Interpretation

Baseline is the speed and simplicity baseline: fewer calls, fewer intermediate artifacts, and lower latency. Its weakness is that the model must invent the cross-user relation and write the final QA in the same response.

Discovery is the debugging and relation-search mode: slower, but it exposes the selected_relation before the MCQ is written. When a question fails answerability, discovery has a better chance to switch to a different relation on retry.

Downstream quality control is identical for both. If discovery produces a bad MCQ, the same reviewer and answerability gates reject it. If baseline produces a strong MCQ, it is accepted by the same final rules.

Minimal command shape:

```
python -m egolife_two_user_qa.cli generate_video_qa_loop --evidence EVIDENCE.jsonl --output qa_mcq.jsonl --prompts-output prompts.jsonl --rejected-output qa_mcq.rejected.jsonl --intermediate-output qa_mcq.intermediate.jsonl --generation-mode discovery
```